RAPPORT DE STAGE BTS Services Informatiques aux Organisations Option SISR – Session 2025

Khelfi Ilyes

Lycée les Chassagnes

BTS SIO 1e année SISR

6 juin au 5 mai

2024 – 2025

Remerciements

Je tiens à remercier toute l'équipe pédagogique de mon établissement pour leur accompagnement tout au long de ma formation, ainsi que mes tuteurs de stage pour leurs disponibilités, conseils et leurs encadrements bienveillant.

Je remercie également les membres de l'équipe informatique avec qui j'ai eu l'opportunité de collaborer, et qui m'ont permis d'approfondir mes compétences dans un environnement professionnel.

Table des matières

RAPPORT DE STAGE BTS Services Informatiques aux Organisations Option SISR – Session 2025 1

Remerciements 2

1. Introduction 4

2. Présentation du contexte du stage 4

L'établissement 4

Le projet 4

Le thème de stage 5

3. Déroulé du projet 5

Réalisations techniques 5

Difficultés rencontrées 6

Organisation et planification 6

Formations complémentaires 7

4. Conclusion 7

5. Annexe 8

Organisation du stage : 8

Fichier. env : 9

Dockerfile : 10

Voici l'interface PhpMyAdmin : 11

Fichier docker-compose.yml : 12

1. Introduction

Dans le cadre de ma deuxième année de BTS SIO, spécialité SISR (Solutions d'Infrastructure, Systèmes et Réseaux), j'ai réalisé un stage de cinq semaines au sein de l'établissement les chassagnes. L'objectif de ce stage était de répondre à un besoin réel de l'établissement en matière de gestion et de suivi des stages des élèves.

Ce stage m'a permis de mettre en œuvre mes compétences techniques en réseau, sécurité, déploiement d'applications web, et d'approfondir mes connaissances dans un environnement professionnel.

2. Présentation du contexte du stage

L'établissement

L'établissement est un lycée qui forme des élèves du secondaire et du supérieur, notamment dans les filières technologiques et professionnelles.

Le projet

Avant mon intervention, la gestion des stages reposait principalement sur les conventions de stage format papier. Cette méthode présentait de nombreuses limites : manque de centralisation, risque de perte de données, absence d'automatisation.

Le projet consistait à déployer une application Laravel existante dans un environnement sécurisé et reproductible à l'aide de Docker, avec les services suivants : serveur PHP, base de données MySQL, serveur web NGINX, interface PhpMyAdmin, pare-feu UFW, certificat HTTPS, et messagerie SMTP.

Le choix des outils Laravel, Docker, NGINX, MySQL avait été validé en amont.

Le thème de stage

"Déploiement d'une plateforme web de suivi de stages avec Laravel, Docker, HTTPS et services associés."

3. Déroulé du projet

Réalisations techniques

- Création du Dockerfile Laravel avec installation des extensions PHP nécessaires.
- Configuration complète du fichier docker-compose.yml avec les conteneurs Laravel, NGINX, MySQL, PhpMyAdmin.
- Génération d'un certificat SSL autosigné avec OpenSSL.
- Configuration d'un hôte local stage.local et mise à jour du fichier /etc/hosts.
- Intégration de PhpMyAdmin derrière un reverse proxy NGINX avec HTTPS.
- Configuration du service SMTP dans le fichier .env de Laravel.
- Mise en place d'une classe Mailable pour tester l'envoi automatique de mails.
- Ajout d'une redirection HTTP vers HTTPS.
- Forçage du HTTPS via Laravel (URL::forceScheme('https')).
- Mise en place d'un pare-feu UFW avec ouverture des ports nécessaires.
- Écriture d'une documentation technique fichier docs/deploiement.md.

Difficultés rencontrées

- Laravel ne se lançait pas à cause d'une erreur de syntaxe dans AppServiceProvider.php. La lecture attentive des logs Docker m'a permis d'identifier et corriger le problème.
- La base de données n'était pas trouvée (SQLSTATE[HY000] [1049] Unknown database). J'ai dû créer manuellement la base ou importer un dump SQL.
- PhpMyAdmin affichait une erreur "File not found" liée à une mauvaise configuration du proxy NGINX.
- L'accès HTTPS échouait avec une erreur de certificat (CN incorrect). J'ai régénéré les certificats avec les bons paramètres.
- L'envoi d'e-mails échouait (STARTTLS mal configuré). Après plusieurs tests, j'ai ajusté le port et l'authentification dans .env.
- Conflit sur le port 8000 : j'ai modifié le port exposé dans docker-compose.yml.
- UFW bloquait les connexions : j'ai ouvert manuellement les ports 8000 (HTTPS) et 22 (SSH).

Organisation et planification

Le projet a été divisé en cinq grandes étapes hebdomadaires :

1. Analyse du besoin et prise en main des sources
2. Création et configuration de l'environnement Docker
3. Mise en place de la sécurité (HTTPS, UFW)
4. Intégration de la messagerie SMTP et tests d'envoi
5. Finalisation, documentation, résolution des derniers bugs

J'ai organisé mes tâches sur Teams, en me fixant des objectifs hebdomadaires. Des points réguliers avec mon tuteur m'ont permis de valider chaque étape.

Formations complémentaires

Pour compléter mes connaissances, j'ai réalisé des recherches sures :

- La configuration avancée de NGINX (reverse proxy, certificats SSL)
- La gestion des services **Docker** avec volumes et réseaux
- L'**envoi de mails** via SMTP avec Laravel
- Les bonnes pratiques de sécurité UFW

4. Conclusion

Ce stage m'a permis de concrétiser les compétences acquises pendant ma formation, et de les appliquer dans un projet réel à fort enjeu pour l'établissement. Le projet est aujourd'hui fonctionnel, stable, et documenté pour être redéployé facilement.

J'ai progressé techniquement (Docker, sécurité, configuration réseau), mais aussi dans ma capacité à résoudre des problèmes complexes, à chercher des solutions, et à travailler de façon autonome.

Ce projet m'a donné confiance pour la suite de mon parcours, et m'a permis d'envisager plus sereinement l'entrée dans le monde professionnel.

5. Annexe

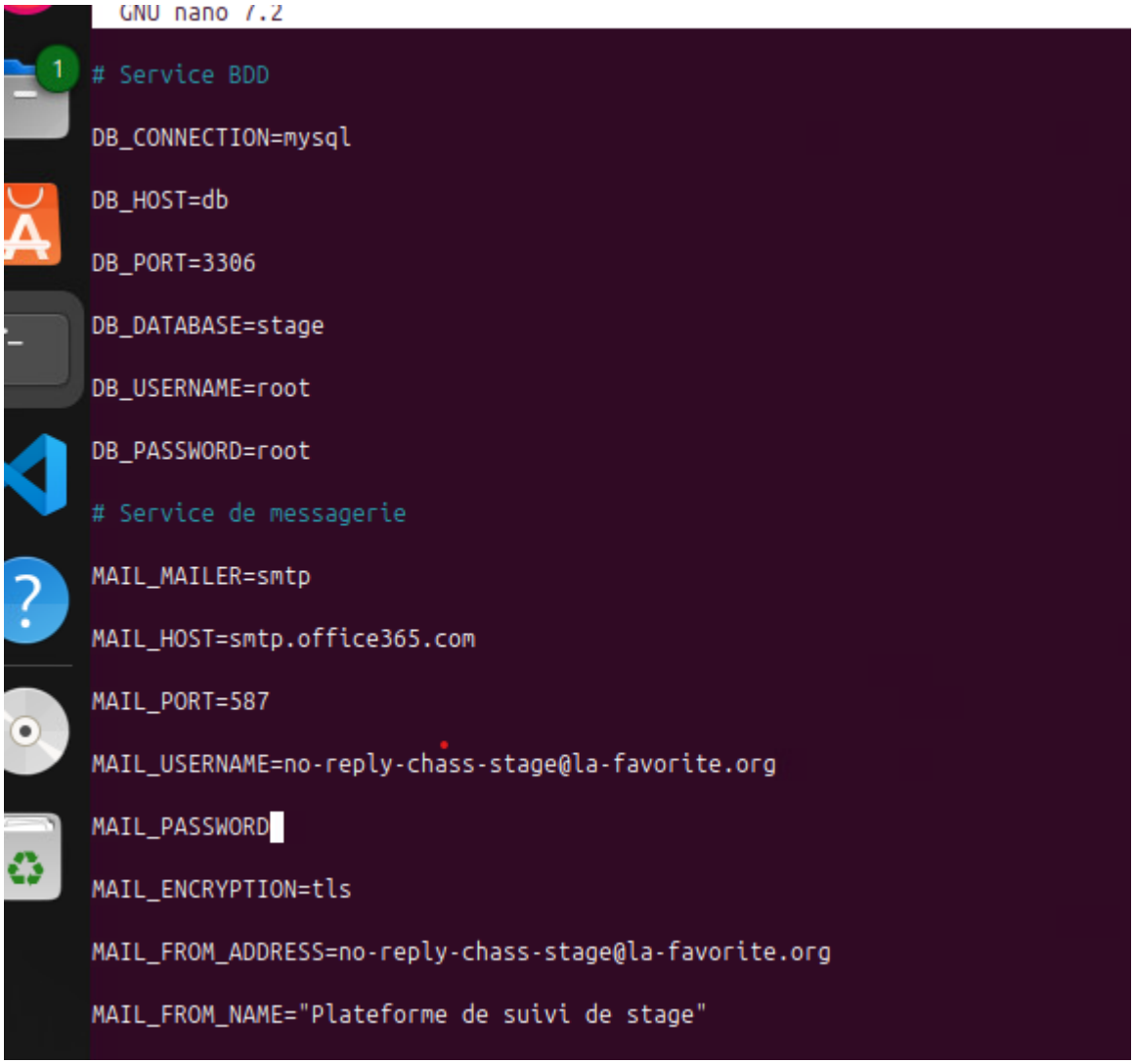
Organisation du stage :

Nom de la tâche	Affectation	Date de début	Date d'échéance	Compartiment	Progression	Priorité	Étiquettes	Aperçu rapide
Rapport d'état mensuel			8/8/2025		Non démarré	Moyen	Ajouter une étiquette	
Effectuer des évaluations sur les performances de l'équij			8/9/2026		Non démarré	Moyen	Ajouter une étiquette	
~~DOCKER~~	KI KHELFI ILYES			recherche docum	Terminées	Moyen	Ajouter une étiquette	
LDAP	FR LN			recherche docum	En cours	Moyen	Ajouter une étiquette	
~~Server Web~~	KI KHELFI ILYES			Service linux	Terminées	Moyen	Ajouter une étiquette	
~~Server BDD~~	KI KHELFI ILYES			Service linux	Terminées	Moyen	Ajouter une étiquette	
~~Plan de sauvegarde~~	KI KHELFI ILYES			Service linux	Terminées	Moyen	Ajouter une étiquette	
~~Firewall~~				Sécurisation	Terminées	Moyen	Ajouter une étiquette	
~~Chiffrement des fichiers~~	LF LAPAIRE FABIE			Sécurisation	Terminées	Moyen	Ajouter une étiquette	
Stockage à sécuriser	LF LAPAIRE FABIE			Sécurisation	Non démarré	Moyen	Ajouter une étiquette	
Suppression des donnés	FR FONSECA RYA			Sécurisation	Non démarré	Moyen	Ajouter une étiquette	
~~Choisir un nom de Domaine~~	FR FONSECA RYA				Terminées	Moyen	Ajouter une étiquette	
~~arborescence~~	FR FONSECA RYA			recherche docum	Terminées	Moyen	Ajouter une étiquette	
~~creation BDD~~	ZE ZOUGS EVAN			recherche docum	Terminées	Moyen	Ajouter une étiquette	
~~maquettes~~	FR FONSECA RYA			recherche docum	Terminées	Moyen	Ajouter une étiquette	
~~formation Laravel MVC~~	FR ZE			recherche docum	Terminées	Moyen	Ajouter une étiquette	
~~formation Docker~~	KI LN			recherche docum	Terminées	Moyen	Ajouter une étiquette	
~~Faille CSRF, Faille XSS, Injection SQL~~	ME MOTHES ENZ(			Sécurisation	Terminées	Moyen	Ajouter une étiquette	
~~Gestion de mail automatique~~				recherche docum	Terminées	Moyen	Ajouter une étiquette	
~~scripts SQL création des tables avec les Expressions régu~~	ME MOTHES ENZ(			Sécurisation	Terminées	Moyen	Ajouter une étiquette	

+ Ajout d'une nouvelle tâche

Fichier. env :

Dans mon projet Laravel, le fichier .env m'a permis de centraliser tous les paramètres sensibles et spécifiques à l'environnement, comme la configuration de la base de données, le serveur mail (SMTP), ou encore l'URL de l'application. Cela facilite la gestion de l'environnement de développement et de production, sans avoir à modifier le code source. Par exemple, j'y ai défini les identifiants Mailtrap pour les tests d'envoi d'e-mails, ainsi que les paramètres de connexion à MySQL.



Dockerfile :

Le Dockerfile m'a servi à créer une image personnalisée pour faire fonctionner Laravel dans un conteneur. J'y ai défini toutes les instructions nécessaires à l'installation de PHP, Composer, les extensions requises (comme ldap, etc.), ainsi que les permissions sur les dossiers storage et bootstrap/cache. Ce fichier permet donc de construire un environnement Laravel sur mesure, cohérent à chaque redéploiement, sans avoir à tout réinstaller manuellement.

```
  GNU nano 7.2
FROM php:8.2-fpm

# Installer les dépendances système
RUN apt-get update && apt-get install -y \
    libldap2-dev \
    build-essential \
    libpng-dev \
    libjpeg-dev \
    libfreetype6-dev \
    zip \
    unzip \
    git \
    curl \
    libonig-dev \
    libxml2-dev \
    libzip-dev \
&& docker-php-ext-install pdo_mysql mbstring zip exif pcntl

# Installer Composer
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

# Créer le dossier de travail dans le conteneur
WORKDIR /var/www

# Copier les fichiers Laravel du projet local dans le conteneur
COPY . .

# Installer les dépendances Laravel
RUN composer install

# Donner les bons droits
RUN chown -R www-data:www-data /var/www \
&& chmod -R 755 /var/www

# Exposer le port (optionnel ici, car nginx fait ça)
EXPOSE 9000
#LDAP

CMD ["php-fpm","-F"]
```

Voici l’interface PhpMyAdmin :

Fichier docker-compose.yml :

Le fichier docker-compose.yml a été essentiel pour orchestrer l'ensemble des services nécessaires à mon application Laravel : le serveur PHP (Laravel), la base de données MySQL, le serveur web NGINX, et PhpMyAdmin. Ce fichier permet de lancer tous les conteneurs en une seule commande, de gérer les réseaux entre eux, les volumes persistants, les ports exposés, et d'assurer une configuration cohérente et reproductible de l'environnement. Cela m'a permis de travailler dans un environnement stable et isolé, quel que soit l'ordinateur utilisé.

```yaml
version: "3.8"

services:

  # Laravel / PHP-FPM

  app:

    build:

      context: .

      dockerfile: Dockerfile

    container_name: laravel_app

    volumes:

      - ./:/var/www

    depends_on:

      - db

    networks:

      - laravel
    expose:
      - "9000"


    #Web Serveur
  webserver:
    image: nginx:alpine
    container_name: nginx_web
    ports:
      - "443:443"
    volumes:
      - ./:/var/www
      - ./docker/nginx/conf.d:/etc/nginx/conf.d
      - ./docker/nginx/certs:/etc/nginx/certs
    depends_on:
      - app
    networks:
      - laravel
    # Base de données MySQL
  db:

    image: mysql:5.7

    container_name: mysql_db

    restart: always

    environment:

      MYSQL_ROOT_PASSWORD: root

      MYSQL_DATABASE: laravel

    volumes:

      - dbdata:/var/lib/mysql

    networks:

      - laravel
```